Lecture 22: Structured Streaming III (Practical)

DSL351 / DSP352: Big Data Analytics

Amit Kumar Dhar

IIT Bhilai

April 13, 2026

Arbitrary Stateful Processing

mapGroupsWithState

- Allows managing custom state for each group/key.
- Input: Key and an Iterator of values for that key in the current batch.
- Output: A single object for that key.
- Use case: User sessionization, state machines.

Custom State Case Classes (Scala)

```scala
case class InputRow(userId: String, eventType: String, timestamp: Long)
case class UserSession(userId: String, count: Int, startTime: Long)
case class OutputRow(userId: String, finalCount: Int)
```

The updateFunction logic

```
def updateFunction(
  userId: String,
  events: Iterator[InputRow],
  state: GroupState[UserSession]): OutputRow = {

  var current = state.getOption.getOrElse(UserSession(userId, 0, Long.MaxValue))

  events.foreach { e =>
    current = current.copy(
      count = current.count + 1,
      startTime = Math.min(current.startTime, e.timestamp)
    )
  }

  state.update(current)
  OutputRow(userId, current.count)
}
```

Applying mapGroupsWithState

```
val streamingQuery = df
  .as[InputRow]
  .groupByKey(_.userId)
  .mapGroupsWithState(GroupStateTimeout.ProcessingTimeTimeout)(updateFunction)
```

flatMapGroupsWithState

- Like `mapGroupsWithState`, but can return multiple rows (an Iterator).
- Allows emitting data when a session times out.
- **Example**: A session ends after 30 minutes of inactivity; emit the session summary only then.

Timeouts in State Management

- **Processing Time Timeout**: `state.setTimeoutDuration("10 minutes")`. Based on system clock.
- **Event Time Timeout**: `state.setTimeoutTimestamp(timestamp)`. Based on watermarks.

Fault Tolerance Deep Dive

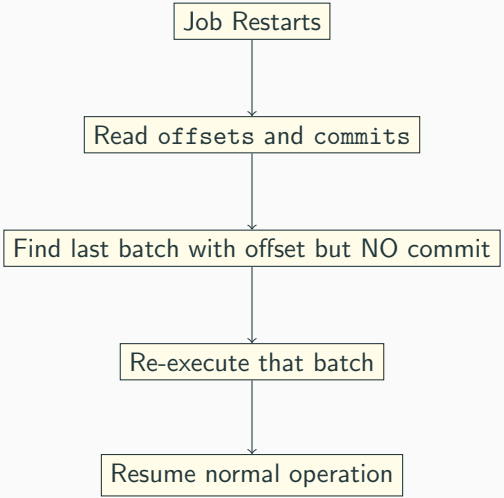
The Exactly-Once Contract

Component	Requirement	Example
Source	Replayable	Kafka, Files
Engine	Deterministic	Spark SQL
Sink	Idempotent	Files, Kafka

Checkpoint Directory Structure

- `offsets/`: WAL of offsets in each micro-batch.
- `commits/`: IDs of batches that successfully finished.
- `state/`: Binary data from aggregations/stateful ops.
- `metadata`: Query ID and other info.

What happens during recovery?



State Store Implementations

- **In-Memory (Default)**: Fast but limited by Java heap size.
- **RocksDB (Recommended for large state)**:
  - Off-heap storage.
  - Better for GBs/TBs of state.
  - Enabled via configuration.

Enabling RocksDB State Store

```
spark.conf.set(
  "spark.sql.streaming.stateStore.providerClass",
  "org.apache.spark.sql.execution.streaming.state.RocksDBStateStoreProvider"
)
```

Monitoring & Performance

The Streaming Tab in Spark UI

Provides real-time metrics:

- **Input Rate**: How many records per second are arriving from the source.
- **Process Rate**: How fast Spark is handling them.
- **Batch Duration**: Time taken for one cycle.

Detecting Performance Bottlenecks

- **Scenario**: Process Rate ¡ Input Rate.
- **Diagnosis**:
  - High Batch Duration? $\rightarrow$ Not enough executors or heavy computation.
  - Skewed partitions? $\rightarrow$ Look at task durations in the Stage tab.
  - Slow sink? $\rightarrow$ Writing to a database with no indexing.

Best Practices: Schema Management

- Always provide a fixed schema for File and Kafka sources in production.
- Use `spark.sql.streaming.schemaInference` cautiously.

Best Practices: Memory Management

- Use Watermarks!
- Monitor `state` memory usage.
- Tune `spark.sql.shuffle.partitions` - default is 200, which is often too high for low-latency small-batch streams.

Combining Streaming and Batch

- **Lambda Architecture**: Serving layer combines Batch (accurate, slow) and Speed (fast, approx).
- **Kappa Architecture**: Use Spark Streaming for both! Just replay history from Kafka.

Advanced Streaming Scenarios

The Small File Problem in Streaming

- Frequent triggers (e.g., every 10s) produce thousands of tiny files in HDFS/S3.
- Tiny files degrade NameNode performance and slow down downstream batch reads.
- **Solution**:
  - Use larger trigger intervals if possible.
  - Periodically run a compaction job to merge small files.
  - Use **Delta Lake** which handles compaction automatically.

Kafka Partitioning & Parallelism

- Spark partitions $\approx$ Kafka partitions.
- If you have 10 Kafka partitions, only 10 Spark tasks will run in parallel for that source.
- **Recommendation**: Over-partition your Kafka topics (e.g., 32 or 64 partitions) to allow for horizontal scaling of Spark executors.

S3 Sink Nuances

- S3 is not a real filesystem (it's an object store).
- Rename operations are expensive.
- Use the **S3A Committer** or **Delta Lake** for reliable and fast writes to S3 from streaming.

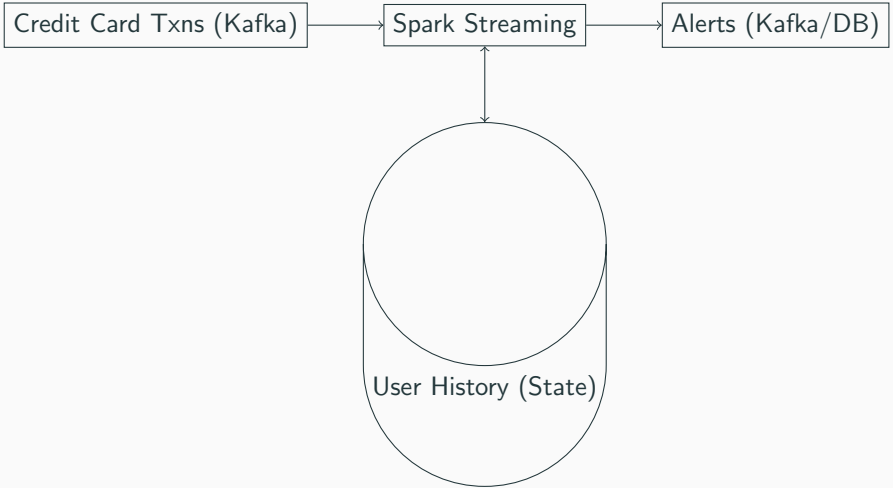
Stream-Stream Join: Multiple Conditions

- You can join on non-equality conditions as long as there is an event-time range.
- Example: "Join if event B happened within 5 minutes after A, AND the location is within 1km".

```
streamA.join(streamB,
  expr("""
    idA = idB AND
    tsB BETWEEN tsA AND tsA + interval 5 minutes AND
    st_distance(locA, locB) < 1000
  """)
)
```

Case Studies

Case Study 1: Real-time Fraud Detection



- **Logic**: Use `mapGroupsWithState` to store last 5 transaction locations.
- **Trigger**: If current txn is$>$ 500km from last one within 10 mins $\rightarrow$ ALERT.

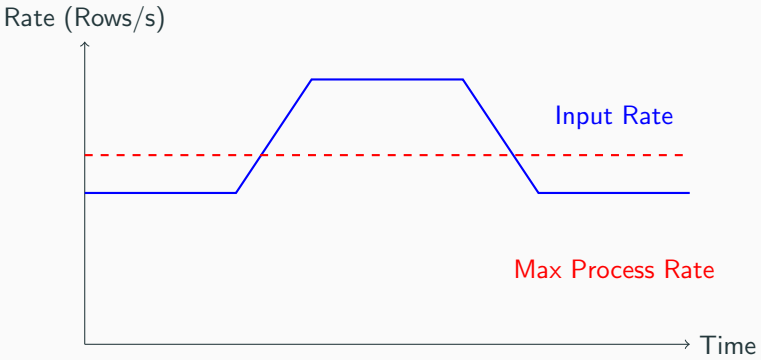
Case Study 2: Ride-Sharing (Uber/Ola Style)

- **Problem**: Match drivers with riders in real-time.
- **Stream 1**: Rider Requests (Location, Timestamp).
- **Stream 2**: Driver Locations (GPS pings).
- **Operation**: Stream-Stream Join with spatial and temporal constraints.
- **Window**: "Match if request and ping are within 2 minutes and 2km".

Case Study 3: Log Analytics Dashboard

- **Goal**: Real-time HTTP 404 error rate per minute.
- **Window**: Tumbling window of 1 minute.
- **Mode**: Update mode to only push changed counts to the dashboard.
- **Watermark**: 5 minutes to handle late logs.

Performance: Input vs Process Rate Visual



Lagging Zone (Input ¿ Process)

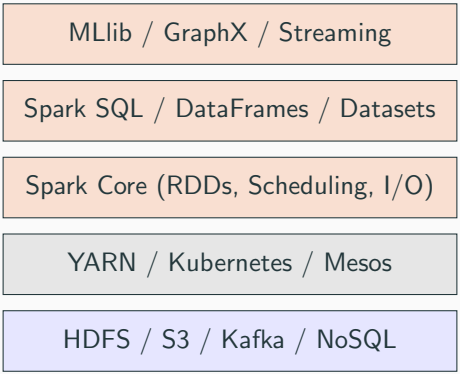
Monitoring: Batch Duration Metrics

- **Stable Query**: `batchDuration` $<$ `triggerInterval`.
- **Unstable Query**: `batchDuration` continues to grow $\rightarrow$ Checkpoint backlog.
- **Resource Tip**: Increase `spark.executor.instances` or use higher core count per executor.

Production Checklist: The "Last Mile"

1. Use **idempotent writes** to avoid duplicates on retry.
2. Configure `maxOffsetsPerTrigger` to prevent memory spikes on restart.
3. Set `spark.sql.streaming.checkpointLocation` to a reliable HDFS/S3 path.
4. Implement **dead-letter queues** (DLQ) for malformed JSON records.

Spark Ecosystem Map



Questions?